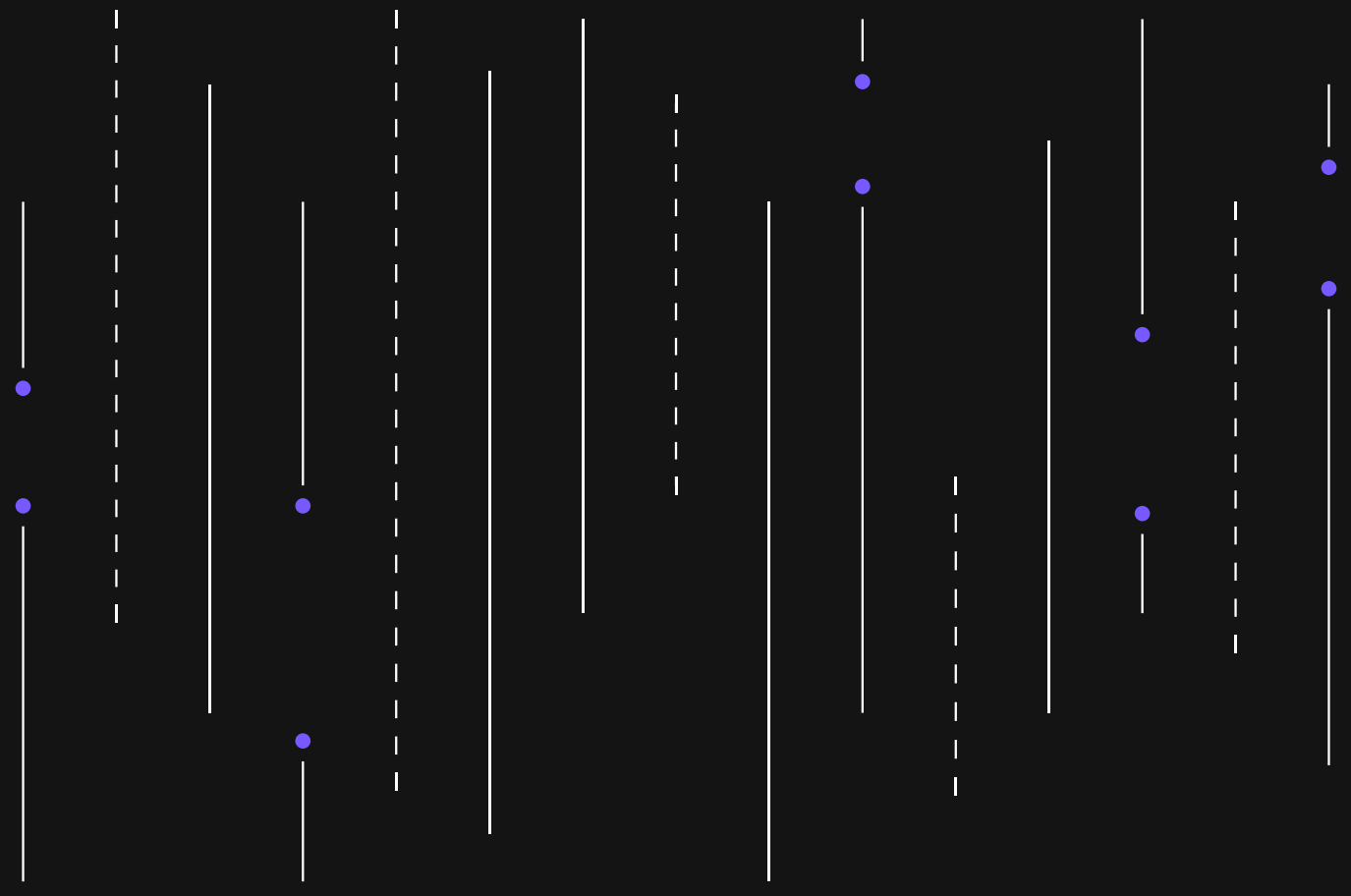
Машинное обучение

Лонгрид

Глубокое обучение

07

Механизм внимания и архитектура трансформера



Содержание

1 Внимание в кодировщике-декодировщике **3**

1.1 Ограничение фиксированного контекстного вектора . . . 3

1.2 Аддитивное внимание . . . 3

1.3 Функции совместимости . . . 4

1.4 Интерпретация весов . . . 4

1.5 Внимание по пространственным позициям . . . 5

2 Архитектура трансформера **5**

2.1 Матричная форма внимания . . . 5

2.2 Многоголовое внимание . . . 6

2.3 Кодировщик трансформера . . . 7

2.4 Декодировщик и причинная маска . . . 9

2.5 Позиционное кодирование . . . 10

2.6 Граф зависимостей и вычислительная стоимость . . . 11

3 Итоги **12**

Механизм внимания и архитектура трансформера

Постановка задачи

Дана входная последовательность $x = (x_1, \dots, x_T)$, где $x_t \in \mathcal{V}$, и выходная последовательность $y = (y_1, \dots, y_S)$. По соглашению недели 6 токену x_t соответствует эмбеддинг $e_t = e(x_t) \in \mathbb{R}^d$; в обозначениях первой части недели 7 это означает $d_e = d$. Кодировщик строит контекстные представления входа, а декодировщик задает условное распределение

$$p_\theta(y \mid x) = \prod_{t=1}^{S} p_\theta(y_t \mid y_{<t}, x).$$

Параметры обучаются минимизацией суммы кросс-энтропий по позициям. Ниже разбираются механизм внимания и исходная архитектура трансформера.

1. Внимание в кодировщике-декодировщике

1.1. Ограничение фиксированного контекстного вектора

В базовом seq2seq-кодировщике вся входная последовательность сжимается в один вектор. Декодировщик не имеет прямого доступа к отдельным позициям входа. При росте длины такой вектор становится узким местом. Внимание заменяет один фиксированный вектор на взвешенную сумму всех состояний кодировщика.

1.2. Аддитивное внимание

Пусть $h_1, \dots, h_T$ - состояния кодировщика, а s_{t-1} - состояние декодировщика перед предсказанием токена y_t. В аддитивном внимании сначала вычисляются оценки совместимости:

$$a_{tj} = v_a^\top \tanh\left(W_s s_{t-1} + W_h h_j + b_a\right).$$

Затем они нормируются по входным позициям:

$$\alpha_{tj} = \frac{\exp a_{tj}}{\sum_{k=1}^{T} \exp a_{tk}}, \qquad c_t = \sum_{j=1}^{T} \alpha_{tj} h_j.$$

Параметры W_s, W_h, v_a, b_a обучаются вместе с остальной сетью. Веса α_{tj} неотрицательны и в сумме по j равны единице, поэтому c_t лежит в выпуклой оболочке состояний кодировщика.

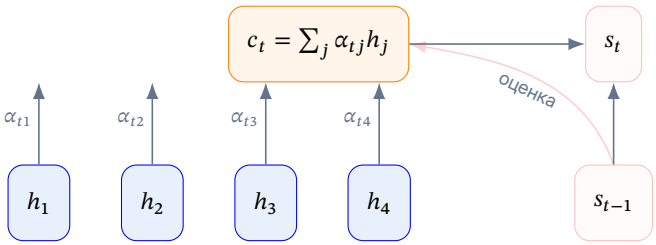


Рис. 1: Контекстный вектор для шага t получается взвешиванием всех состояний кодировщика.

1.3. Функции совместимости

Веса внимания определяются оценкой совместимости запроса q и ключа k. В аддитивном внимании используется обучаемая функция

$$\text{score}_{\text{add}}(q, k) = v_a^{\top} \tanh\left(W_q q + W_{\ k} k + b_a\right).$$

Другой вариант - скалярное произведение

$$\text{score}_{\text{dot}}(q, k) = q^{\top} k.$$

Аддитивная функция сначала переводит запрос и ключ в общее пространство. Скалярное произведение требует одинаковой размерности, зато оценки для многих запросов и ключей вычисляются одним матричным умножением. В трансформере используется масштабированное скалярное произведение

$$\text{score}_{\text{scaled}}(q, k) = \frac{q^{\top} k}{\sqrt{d_k}}.$$

Множитель $1/\sqrt{d_k}$ контролирует масштаб оценок перед softmax. Его роль и матричная запись внимания рассматриваются ниже.

1.4. Интерпретация весов

Матрица весов показывает, какие позиции входа сильнее участвовали в вычислении конкретного выхода. Эта визуализация полезна для анализа модели, но сама по себе не дает причинного объяснения прогноза.

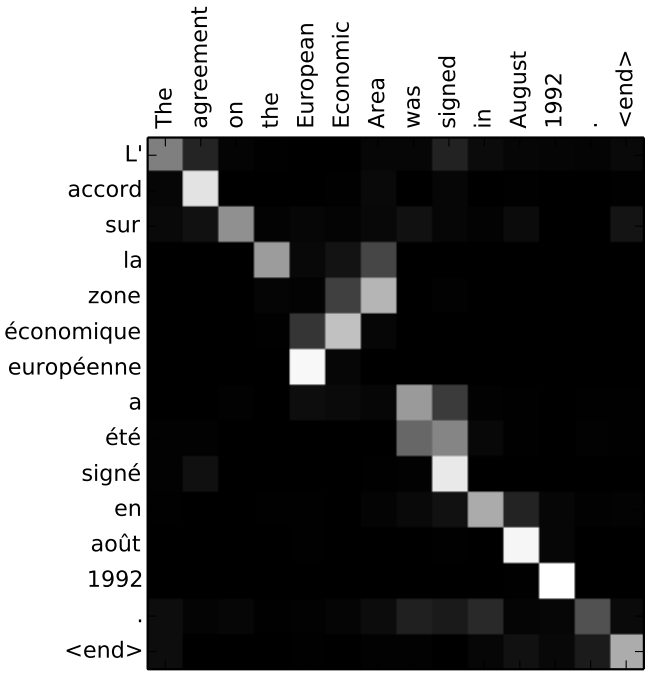


Рис. 2: Веса внимания для перевода с английского на французский. Перестановка слов в European Economic Area отражается в недиагональном фрагменте матрицы. Источник: Bahdanau et al., 2015, Figure 3(a) [2].

1.5. Внимание по пространственным позициям

В задаче порождения подписи к изображению ключами и значениями становятся пространственные признаки сверточной сети. На каждом шаге декодировщик выбирает свою взвешенную сумму этих признаков.

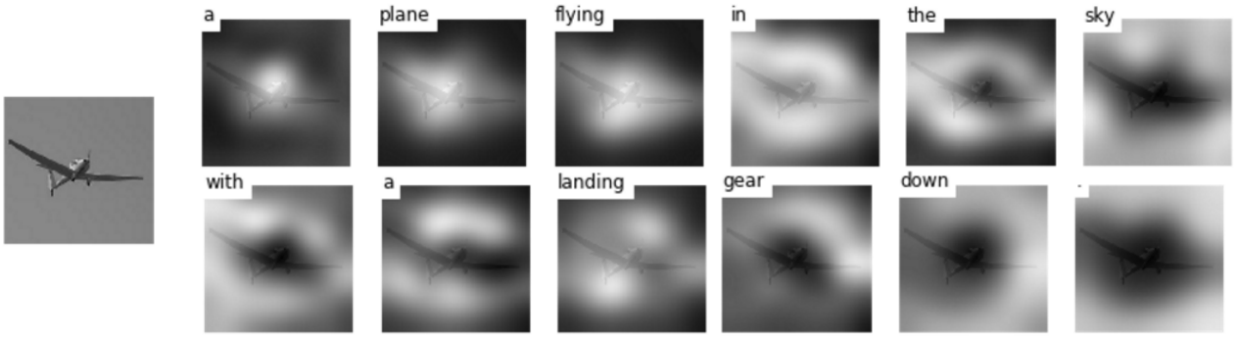


Рис. 3: Пространственные веса внимания при генерации подписи *a plane flying in the sky with a landing gear down*. Светлые области соответствуют большим весам. Источник: Xu et al., 2015, Figure 2 [3].

2. Архитектура трансформера

Трансформер заменяет рекуррентные переходы матричным вниманием. Все позиции одного слоя можно вычислить параллельно. Порядок элементов задается отдельно позиционными векторами.

2.1. Матричная форма внимания

Рекуррентная сеть вычисляет состояние h_t после состояния h_{t-1}. Это последовательная зависимость самих вычислений: все позиции одного слоя нельзя обработать независимо. В трансформере связь между позициями организована через внимание. Оно позволяет одновременно вычислить выходы слоя для всех доступных позиций, хотя каждый выход по-прежнему зависит от других элементов.

Для внимания нужны три объекта: запросы, ключи и значения. Запрос определяет, какую информацию ищет данная позиция. Ключи используются для оценки совместимости с запросом. Значения - векторы, которые суммируются с полученными весами. Ключ и значение одного элемента могут быть разными преобразованиями исходного состояния.

Соберем векторы в строки матриц:

$$Q \in \mathbb{R}^{n_q \times d_k}, \qquad K \in \mathbb{R}^{n_k \times d_k}, \qquad V \in \mathbb{R}^{n_k \times d_v}.$$

Число запросов n_q может отличаться от числа ключей n_k. Размерности запроса и ключа одинаковы для скалярного произведения. Значения имеют собственную размерность d_v. В матрице $QK^\top$ элемент (i, j) равен произведению запроса q_i на ключ k_j.

Масштабированное скалярное внимание (scaled dot-product attention) записывается как

$$A = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \qquad O = \mathrm{Attention}(Q, K, V) = AV.$$

Softmax берется по каждой строке, то есть по ключам для одного запроса. Получаем $A \in \mathbb{R}^{n_q \times n_k}$, где

веса неотрицательны и сумма каждой строки равна единице. Затем

$$O \in \mathbb{R}^{n_q \times d_v}, \qquad o_i = \sum_{j=1}^{n_k} A_{ij} v_j.$$

Таким образом, каждому запросу соответствует собственная взвешенная сумма значений. Это матричная запись уже знакомого контекстного вектора.

Для связи с рекуррентным вниманием рассмотрим запрос $\widetilde{h}_i$ декодировщика и состояния h_t кодировщика. Без масштабирования контекстный вектор записывается поэлементно:

$$c_i = \sum_{t=1}^{T} \frac{\exp(\widetilde{h}_i^\top h_t)}{\sum_{j=1}^{T} \exp(\widetilde{h}_i^\top h_j)} h_t.$$

Здесь i нумерует запросы выходной последовательности, t и j - позиции входа. В матричной записи строки Q соответствуют запросам $\widetilde{h}_i$, строки K - ключам h_t, а строки V - суммируемым значениям. В более общем случае ключи и значения получают разными проекциями.

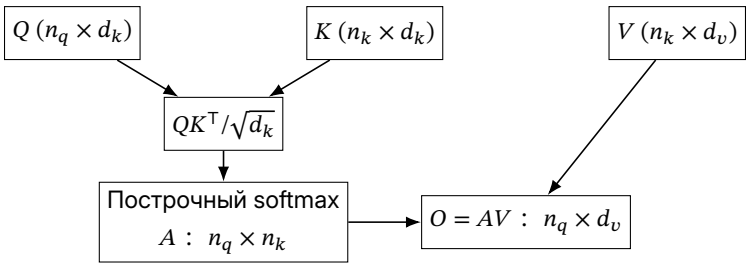


Рис. 4: Размерности запросов, ключей, значений и результата внимания [1].

Деление на $\sqrt{d_k}$ контролирует масштаб оценок. Если координаты запроса и ключа независимы, имеют нулевое среднее и единичную дисперсию, то дисперсия их скалярного произведения равна d_k. После деления она равна единице. Без такой нормировки при большой размерности softmax может становиться слишком резким. Предположения объясняют выбор масштаба, но не означают, что реальные обученные векторы обязаны быть независимыми.

В самой операции Attention нет обучаемых матриц: она работает с переданными Q, K, V. Обучаемыми становятся преобразования, которые создают эти матрицы из состояний сети.

2.2. Многоголовое внимание

Одна система оценок совместимости может быть недостаточно гибкой. Многоголовое внимание вычисляет несколько операций внимания с различными проекциями [1]. Обозначим входные матрицы через

$$X_Q \in \mathbb{R}^{n_q \times d}, \qquad X_K, X_V \in \mathbb{R}^{n_k \times d}.$$

Для головы r введем

$$W_r^Q, W_r^K \in \mathbb{R}^{d \times d_k}, \qquad W_r^V \in \mathbb{R}^{d \times d_v}.$$

Ее запросы, ключи и значения равны

$$Q_r = X_Q W_r^Q, \qquad K_r = X_K W_r^K, \qquad V_r = X_V W_r^V.$$

Обозначим число голов через h. Каждая голова вычисляет

$$\text{head}_r = \text{Attention}(Q_r, K_r, V_r), \qquad r = 1, \ldots, h.$$

Результат одной головы имеет размер $n_q \times d_v$. Конкатенируем результаты по координатам признаков и преобразуем их общей выходной матрицей:

$$\mathrm{MHA}(X_Q, X_K, X_V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W^O, \qquad W^O \in \mathbb{R}^{hd_v \times d}.$$

Конкатенация выполняется по координатам признаков. Итог снова имеет размер $n_q \times d$, удобный для остаточной связи с входом.

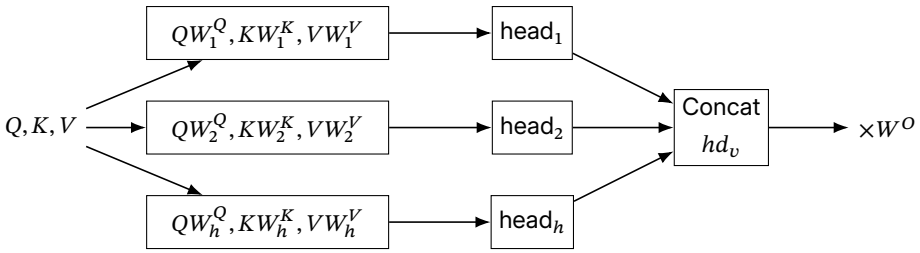


Рис. 5: Несколько проекций внимания и объединение результатов голов [1].

Матрицы проекций и W^O обучаются градиентом конечной задачи. Для исходного базового трансформера используются $d = 512$, $h = 8$, $d_k = d_v = 64$. Каждая голова работает в меньшей размерности, а восемь результатов вместе дают 512 координат перед выходной проекцией.

Разные головы могут изучить разные отношения между элементами. Однако конкретной голове не назначают заранее роль «синтаксис» или «семантика», и различие голов не гарантируется одним их количеством. Это несколько независимо параметризованных способов извлечь информацию из одной последовательности.

2.3. Кодировщик трансформера

В задаче перевода дано исходное предложение $x_1, \ldots, x_T$. Кодировщик должен получить для каждой позиции контекстное состояние; декодировщик затем использует весь набор состояний. В строке t входной матрицы стоит представление токена $e_t = e(x_t) \in \mathbb{R}^d$, введенное на неделе 6. Обозначим матрицу входных представлений через $H^{(0)} \in \mathbb{R}^{T \times d}$; ниже к эмбеддингам токенов добавим позиционные векторы.

В самовнимании (self-attention) запросы, ключи и значения получаются из одной и той же последовательности. Поэтому первый подслой блока вычисляет $\mathrm{MHA}(H, H, H)$. Для каждой позиции он оценивает совместимость со всеми доступными позициями, включая себя, и суммирует их значения. Так токен получает информацию о контексте.

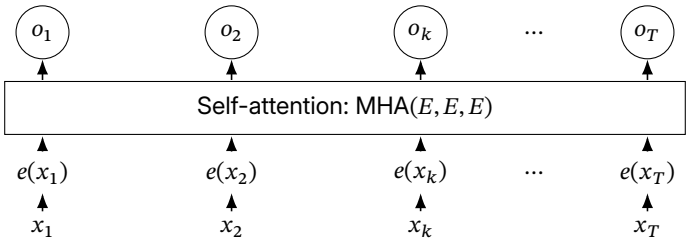


Рис. 6: Самовнимание связывает позиции одной входной последовательности [1].

Второй подслой - полносвязная сеть, которая применяется к каждой позиции с одинаковыми параметрами. При записи векторов строками она имеет вид

$$\mathrm{FFN}(H) = \mathrm{ReLU}(HW_1 + b_1)W_2 + b_2,$$

где $W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$, а смещения добавляются к каждой строке. Первый линейный слой меняет размерность признаков, нелинейность преобразует их, второй возвращает размерность d. FFN не смешивает позиции; смешивание произошло в подслое внимания.

Вокруг каждого подслоя стоит остаточная связь, после сложения выполняется LayerNorm. Один блок исходной архитектуры можно записать так:

$$U^{(\ell)} = \text{LayerNorm}\left(H^{(\ell-1)} + \text{MHA}_\ell(H^{(\ell-1)}, H^{(\ell-1)}, H^{(\ell-1)})\right),$$
$$H^{(\ell)} = \text{LayerNorm}\left(U^{(\ell)} + \text{FFN}_\ell(U^{(\ell)})\right), \qquad \ell = 1, \dots, L.$$

У разных блоков свои параметры. Размер матриц $H^{(\ell)}$ сохраняется: $T \times d$. Остаточная связь дает прямой путь для информации и градиента, а FFN и внимание могут обучать поправки к входному представлению. Формулы показывают исходный порядок нормализации после сложения, называемый post-norm. Dropout, применяемый при обучении исходной модели, здесь опущен для ясности записи.

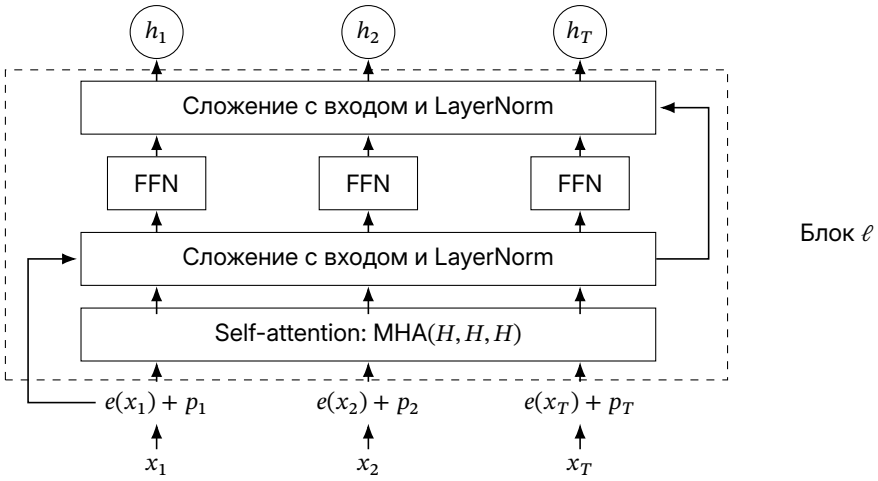


Рис. 7: Блок кодировщика: внимание, сеть по позициям и две остаточные связи [1].

Послойная нормализация (LayerNorm) нормирует признаки одного токена. Если $h \in \mathbb{R}^d$ - его вектор, то

$$\mu = \frac{1}{d}\sum_{j=1}^{d} h_j, \qquad \nu = \frac{1}{d}\sum_{j=1}^{d}(h_j - \mu)^2, \qquad \text{LayerNorm}(h)_j = \gamma_j \frac{h_j - \mu}{\sqrt{\nu + \epsilon}} + \beta_j.$$

Параметры γ_j, β_j обучаются, $\epsilon > 0$ обеспечивает устойчивость. Среднее и дисперсия считаются по координатам текущего вектора, отдельно для каждой позиции и каждого примера. Статистика других элементов батча не нужна. Это полезно для последовательностей разной длины и не создает дополнительного обмена информацией между позициями.

Все запросы одного слоя вычисляются матрично. Поэтому кодировщику не нужен последовательный цикл, в котором состояние позиции t строится из только что вычисленного состояния $t - 1$. Последовательность слоев остается: следующий блок получает результат предыдущего. Параллельность по позициям не означает, что слова обрабатываются без учета друг друга.

В исходном базовом трансформере кодировщик состоит из шести блоков, а размерность состояния равна 512 [1].

2.4. Декодировщик и причинная маска

Для перевода нужно породить новую последовательность $y_1, \dots, y_S$. Поэтому декодировщик должен сочетать информацию об исходном предложении и уже известном префиксе ответа. Его вход при обучении с правильным префиксом (teacher forcing) сдвинут на один токен:

$$z_1 = \texttt{<bos>}, \qquad z_t = y^*_{t-1}, \quad t = 2, \dots, S.$$

Выход в позиции t предсказывает y^*_t. Если в обучении дать ему доступ к будущим позициям входа декодировщика, он увидит там правильный ответ. В частности, в позиции $t+1$ на входе уже стоит y^*_t. Такую связь нужно исключить.

Добавим к матрице оценок внимания маску:

$$\text{Attention}_M(Q, K, V) = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Для самовнимания декодировщика $M \in \mathbb{R}^{S \times S}$ задается как

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

В нижнем треугольнике и на диагонали стоят нули. Верхний треугольник закрыт. После softmax запрещенные позиции имеют нулевой вес. Диагональ разрешена, потому что текущий вход z_t содержит предыдущий выходной токен, а не предсказываемый y_t.

Для четырех позиций это маска

$$M = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

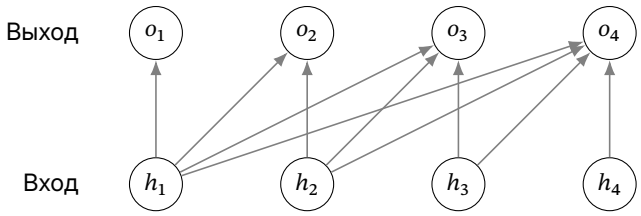


Рис. 8: Разрешенные связи причинного внимания: выход i видит только входы $j \le i$.

Такая маска называется причинной (causal mask): состояние не зависит от будущих токенов. В реализации вместо $-\infty$ используют корректную для выбранного числового формата операцию маскирования или достаточно большое по модулю отрицательное число. Отдельной маской исключают позиции заполнения, добавленные для выравнивания длин в батче.

Теперь обозначим выход кодировщика через $H^{(L)}$, а начальные состояния декодировщика через $D^{(0)}$. У каждого блока декодировщика три подслоя:

$$\begin{aligned} U^{(\ell)} &= \text{LayerNorm}\left(D^{(\ell-1)} + \text{MHA}^{\text{causal}}_{\ell}(D^{(\ell-1)}, D^{(\ell-1)}, D^{(\ell-1)})\right), \\ C^{(\ell)} &= \text{LayerNorm}\left(U^{(\ell)} + \text{MHA}^{\text{cross}}_{\ell}(U^{(\ell)}, H^{(L)}, H^{(L)})\right), \\ D^{(\ell)} &= \text{LayerNorm}\left(C^{(\ell)} + \text{FFN}_{\ell}(C^{(\ell)})\right). \end{aligned}$$

Первый подслой использует только доступный выходной префикс. Во втором запросы приходят из декодировщика, а ключи и значения - из кодировщика. Это перекрестное внимание (cross-attention). Для каждого выходного токена оно выбирает информацию из исходного предложения. Причинная маска по позициям исходного предложения здесь не нужна: исходный текст известен целиком. Третий подслой преобразует признаки каждой позиции независимо.

Из последнего слоя получают распределение по выходному словарю. Для вектора-строки $D_t^{(L)}$ это

$$p_\theta(y_t = a \mid y_{<t}, x) = \mathrm{softmax}(D_t^{(L)} W_{\text{out}} + b_{\text{out}})_a,$$

где $W_{\text{out}} \in \mathbb{R}^{d \times V_{\text{out}}}$ и $b_{\text{out}} \in \mathbb{R}^{V_{\text{out}}}$ обучаются. При обучении с правильным префиксом минимизируется

$$\mathcal{L}_{\text{CE}}(x, y^*) = -\sum_{t=1}^{S} \log p_\theta(y_t^* \mid y_{<t}^*, x).$$

Все правильные префиксы известны, поэтому причинная маска позволяет считать выходы по позициям параллельно.

При генерации новые токены появляются последовательно. Сначала доступен только <bos>, затем первый прогноз, затем второй. Трансформер не устраняет авторегрессионный характер генерации. Кеширование уже вычисленных ключей и значений уменьшает повторные вычисления, но не позволяет заранее узнать еще не выбранный токен. Для выбора ответа по-прежнему можно применять жадный или лучевой поиск.

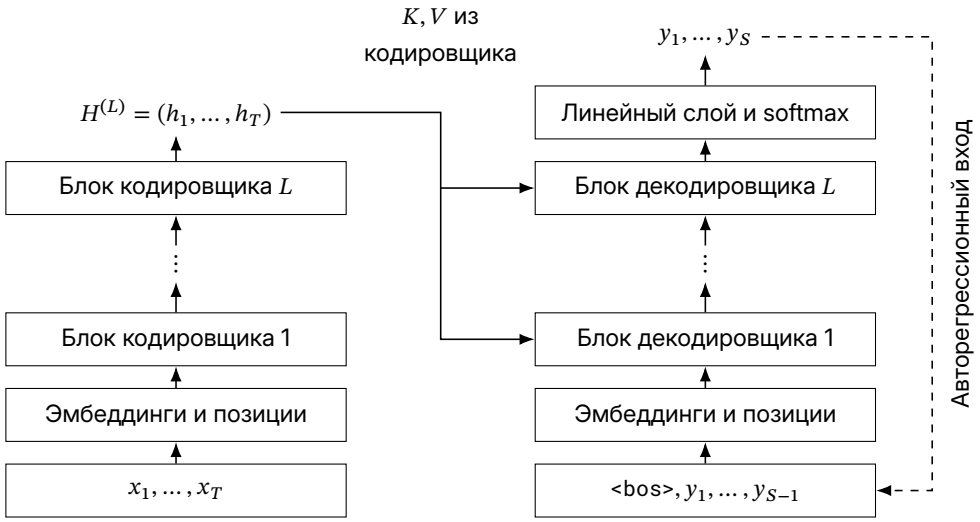


Рис. 9: Стек кодировщика и декодировщика. Каждый блок декодировщика получает ключи и значения из последнего слоя кодировщика [1].

2.5. Позиционное кодирование

Описанный двунаправленный кодировщик без позиционной информации не различает порядок одинакового набора токенов. Точное свойство здесь - эквивариантность. Если переставить строки входа, строки выхода переставятся тем же способом. Это не означает, что выходная матрица останется неизменной.

Пусть $P \in \mathbb{R}^{T \times T}$ - матрица перестановки. Для немаскированного самовнимания без позиционных признаков

$$F(PH) = PF(H).$$

Действительно, перестановка меняет матрицу оценок как $QK^\top \mapsto P(QK^\top)P^\top$. Построчный softmax переставляет соответствующие веса, а умножение на PV дает ту же перестановку выхода. Преобразования

каждой строки и LayerNorm также сохраняют это свойство. Инвариантность возникает, например, после симметричного усреднения строк, когда получается один вектор для всего множества.

Для текста порядок обычно существенен. Предложения «собака укусила человека» и «человек укусил собаку» состоят из одинаковых слов, но имеют разный смысл. Чтобы кодировщик различал их, к вектору токена добавляют вектор позиции:

$$H_t^{(0)} = \sqrt{d}\, e(x_t) + p_t.$$

Оба вектора имеют размерность d. Множитель $\sqrt{d}$ соответствует исходной реализации трансформера [1].

В исходном трансформере используется синусоидальное кодирование. При четном d и индексе пары координат $i = 0, \dots, d/2 - 1$

$$\omega_i = 10000^{-2i/d}, \qquad p_{t,2i} = \sin(\omega_i t), \qquad p_{t,2i+1} = \cos(\omega_i t).$$

Позиция t задает аргумент, а частота ω_i - масштаб изменения пары координат. Одни компоненты меняются быстрее, другие медленнее. Вектор позиции в этом варианте вычисляется по формуле, а не обучается как отдельная таблица.

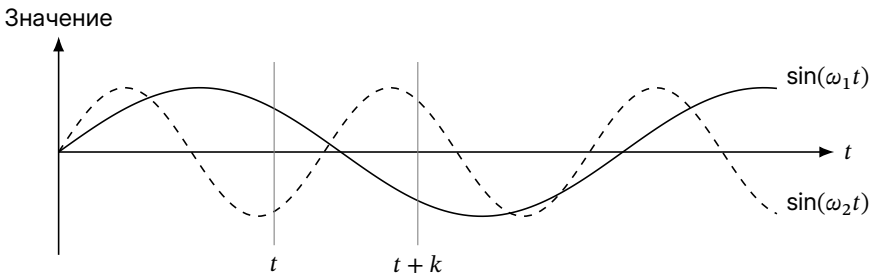


Рис. 10: Синусоидальные компоненты с разными частотами. Вертикальные сечения задают две позиции [1].

Для одной синусоидальной компоненты формула сложения дает

$$\sin(\omega_i(t+k)) = \sin(\omega_i t)\cos(\omega_i k) + \cos(\omega_i t)\sin(\omega_i k).$$

Коэффициенты при координатах позиции t зависят только от смещения k и выбранной частоты. Косинус преобразуется аналогично.

Мотивацию можно увидеть из формул сложения. Для фиксированного смещения k

$$\begin{pmatrix} \sin(\omega_i(t+k)) \\ \cos(\omega_i(t+k)) \end{pmatrix} = \begin{pmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{pmatrix} \begin{pmatrix} \sin(\omega_i t) \\ \cos(\omega_i t) \end{pmatrix}.$$

Сдвиг на k выражается линейным преобразованием пары координат, причем коэффициенты не зависят от исходной позиции t. Это дает модели удобную структуру для учета относительного расположения. Формула не гарантирует успешную работу на сколь угодно длинных последовательностях.

В кодировщике-декодировщике позиционная информация нужна обеим последовательностям.

2.6. Граф зависимостей и вычислительная стоимость

В рекуррентном слое путь между двумя далекими позициями проходит через промежуточные состояния. В полном самовнимании любая пара позиций связана за один слой. Плата за это - матрица весов размера $T \times T$.

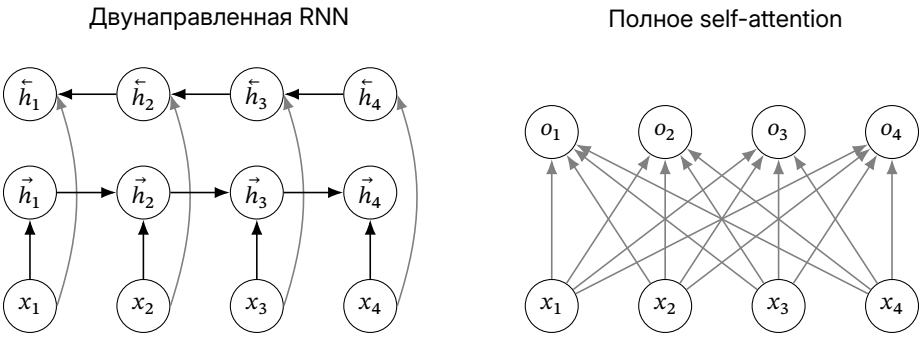


Рис. 11: Зависимости в двунаправленной RNN и в полном самовнимании.

Для одного блока с состояниями размерности d обучаемые проекции требуют $O(Td^2)$ операций, вычисление и применение матрицы весов - $O(T^2d)$, а FFN - $O(Td\,d_{\text{ff}})$. Матрица весов занимает $O(T^2)$ памяти на голову. При $hd_k = hd_v = d$ многоголовое внимание имеет тот же порядок вычислительной стоимости, что и одна голова ширины d.

3. Итоги

Внимание дает каждому запросу свою взвешенную сумму значений. В многоголовом внимании запросы, ключи и значения строятся обучаемыми проекциями. Кодировщик трансформера чередует самовнимание и FFN, а декодировщик добавляет причинную маску и перекрестное внимание. Позиционные векторы возвращают в модель информацию о порядке. Параллельность по позициям ускоряет обучение, но полное внимание требует квадратичной памяти по длине последовательности.

Список литературы

[1] Vaswani A. et al. Attention Is All You Need. 2017. arXiv:1706.03762.

[2] Bahdanau D., Cho K., Bengio Y. Neural Machine Translation by Jointly Learning to Align and Translate. 2015. arXiv:1409.0473.

[3] Xu K. et al. Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. 2015. arXiv:1502.03044.